

LISTE, PILE E CODE

PIETRO DI LENA

DIPARTIMENTO DI INFORMATICA – SCIENZA E INGEGNERIA
UNIVERSITÀ DI BOLOGNA

ALGORITMI E STRUTTURE DI DATI
ANNO ACCADEMICO 2024/2025

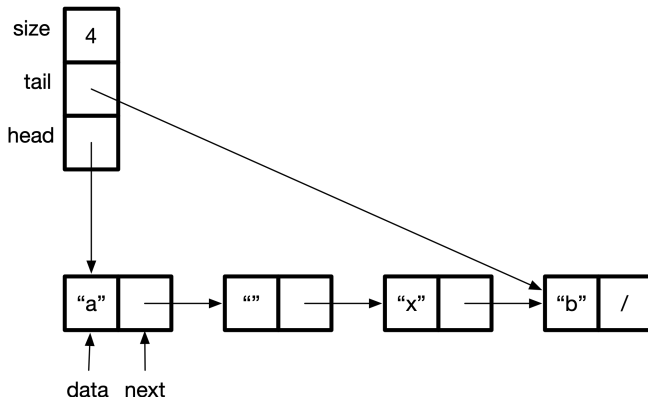


ESERCIZIO

- Implementare la seguente interfaccia utilizzando una rappresentazione di tipo *lista concatenata con puntatore alla testa e alla coda*

```
1 public interface List<D> {  
2     // Inserts data at the head of the list  
3     public void insert(D data);  
4     // Inserts data at the tail of the list  
5     public void append(D data);  
6     // Returns true if the list contains data  
7     public boolean search(D data);  
8     // Returns the data at the specified position  
9     public D get(int i) throws IndexOutOfBoundsException;  
10    // Replaces the data at position i with data. Returns old data  
11    public D set(int i, D data) throws IndexOutOfBoundsException;  
12    // Removes the first occurrence of data  
13    public void delete(D data);  
14    // Returns true if the list is empty  
15    public boolean isempty();  
16    // Returns the number of elements in the list  
17    public int size();  
18 }
```

ORGANIZZAZIONE DI UNA DOUBLEENDLIST



- Chiamiamo la nostra classe *DoubleEndedList*
- Una lista è composta da una catena di nodi, da un puntatore alla testa (*head*) e da uno alla coda (*tail*) della catena
- La classe vi viene fornita già parzialmente implementata

PSEUDOCODICE DI SEARCH

```
1: function SEARCH(DELIST  $L$ , DATA  $d$ )  $\rightarrow$  BOOL  
2:    $tmp = L.head$   
3:   while  $tmp \neq \text{NIL}$  do  
4:     if  $tmp.data == d$  then  
5:       return true  
6:      $tmp = tmp.next$   
7:   return false
```

- Il confronto a riga 4 presuppone che gli oggetti do tipo *Data* siano confrontabili rispetto all'operatore di uguaglianza `==`
- Quale metodo potremmo utilizzare?

PSEUDOCODICE DI INSERT E APPEND

```
1: function INSERT(DELIST L, DATA d)
2:   tmp = NODE(d)
3:   if L.head == NIL then
4:     L.head = L.tail = tmp
5:   else
6:     tmp.next = L.head
7:     L.head = tmp
8:   L.size = L.size + 1
```

```
1: function APPEND(DELIST L, DATA d)
2:   tmp = NODE(d)
3:   if L.head == NIL then
4:     L.head = L.tail = tmp
5:   else
6:     L.tail.next = tmp
7:     L.tail = tmp
8:   L.size = L.size + 1
```

PSEUDOCODICE DI GET

```
1: function GET(DELIST  $L$ , INT  $i$ )  $\rightarrow$  DATA  
2:   if  $i < 1$  or  $i > L.size$  then  
3:     return NIL  
4:    $tmp = L.head$   
5:   while  $i > 1$  do  
6:      $tmp = tmp.next$   
7:      $i = i - 1$   
8:   return  $tmp.data$ 
```

- L'implementazione Java deve lanciare un'eccezione a riga 3
- Attenzione all'indicizzazione: nello pseudocodice utilizziamo *1-based indexing* mentre nell'implementazione *0-based indexing*

PSEUDOCODICE DI SET

```
1: function SET(DELIST  $L$ , INT  $i$ , DATA  $D$ )  $\rightarrow$  DATA  
2:   if  $i < 1$  or  $i > L.size$  then  
3:     return NIL  
4:    $tmp = L.head$   
5:   while  $i > 1$  do  
6:      $tmp = tmp.next$   
7:      $i = i - 1$   
8:    $olddata = tmp.data$   
9:    $tmp.data = data$   
10:  return  $olddata$ 
```

- L'implementazione Java deve lanciare un'eccezione a riga 3
- Attenzione all'indicizzazione: nello pseudocodice utilizziamo *1-based indexing* mentre nell'implementazione *0-based indexing*

PSEUDOCODICE DI DELETE, SIZE E ISEMPY

```
1: function DELETE(DELIST L, DATA d)
2:   prev = NIL, curr = L.head
3:   while curr ≠ NIL and d ≠ curr.data do
4:     prev = curr
5:     curr = curr.next
6:   if curr ≠ NIL then
7:     if curr == L.head then
8:       L.head = curr.next
9:     else
10:      prev.next = curr.next
11:    if curr == L.tail then
12:      L.tail = prev
13:    L.size = L.size - 1
```

```
1: function SIZE(DELIST L) → INT
2:   return L.size
```

```
1: function ISEMPY(DELIST L) → BOOL
2:   return L.size == 0
```


ESERCIZIO

- Implementare l'interfaccia *Stack* definita di seguito

```
1 import java.util.NoSuchElementException;
2 public interface Stack<D> {
3     // Pushes data on top of the stack
4     public void push(D data);
5
6     // Removes and returns the data on top of the stack
7     public D pop() throws NoSuchElementException;
8
9     // Returns the data on top of the stack
10    public D top() throws NoSuchElementException;
11
12    // Returns true if the stack is empty
13    public boolean isempty();
14
15    // Returns the number of elements in the stack
16    public int size();
17 }
```

- Modificare *DoubleEndedList* in modo che implementi anche questa nuova interfaccia *Stack* (abbiamo già *isempty()* e *size()*)

PSEUDOCODICE RISPETTO ALLE OPERAZIONI LISTA

```
1: function PUSH(DELIST  $L$ , DATA  $d$ )  
2:   INSERT( $L$ ,  $d$ );
```

```
1: function POP(DELIST  $L$ )  $\rightarrow$  DATA  
2:   if  $L.size == 0$  then return NIL  
3:   DATA  $d =$  GET( $L$ , 1)  
4:   DELETE( $L$ ,  $d$ )  
5:   return  $d$ 
```

```
1: function TOP(DELIST  $L$ )  $\rightarrow$  DATA  
2:   if  $L.size == 0$  then return NIL  
3:   return GET( $L$ , 1)
```

- INSERT, DELETE, GET sono le operazioni già sviluppate nella classe
- La classe Java deve ritornare un'eccezione in POP e TOP (non *null*)
- Non è necessario reimplementare IEMPTY e SIZE

- Implementare l'interfaccia *Queue* definita di seguito

```
1 import java.util.NoSuchElementException;
2
3 public interface Queue<D> {
4     // Inserts data into the queue
5     public void enqueue(D data);
6
7     // Removes and returns the first data in the queue
8     public D dequeue() throws NoSuchElementException;
9
10    // Returns the first data in the queue
11    public D first() throws NoSuchElementException;
12
13    // Returns true if the queue is empty
14    public boolean isEmpty();
15
16    // Returns the number of elements in the queue
17    public int size();
18 }
```

- Come prima, modificare *DoubleEndedList*

PSEUDOCODICE RISPETTO A OPERAZIONI LISTA/PILA

```
1: function ENQUEUE(DELIST  $L$ , DATA  $d$ )  
2:   APPEND( $L, d$ );
```

```
1: function DEQUEUE(DELIST  $L$ )  $\rightarrow$  DATA  
2:   return POP( $L$ )
```

```
1: function FIRST(DELIST  $L$ )  $\rightarrow$  DATA  
2:   return TOP( $L$ )
```

- Come prima, ISEMPY e SIZE non vanno reimplementate